Solution: Middle of the LinkedList

Problem Statement

Given the head of a **Singly LinkedList**, write a method to return the **middle node** of the LinkedList.

If the total number of nodes in the LinkedList is even, return the second middle node.

Example 1:

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> null
Output: 3
```

Example 2:

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
Output: 4
```

Example 3:

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> null
Output: 4
```

Solution

One brute force strategy could be to first count the number of nodes in the LinkedList and then find the middle node in the second iteration. Can we do this in one iteration?

We can use the **Fast** & **Slow pointers** method such that the fast pointer is always twice the nodes ahead of the slow pointer. This way, when the fast pointer reaches the end of the LinkedList, the slow pointer will be pointing at the middle node.

let's visualize example 3 via the below diagram.

Code

Here is what our algorithm will look like:

Java

```java
/*class ListNode {
  int val = 0;
  ListNode next;

  ListNode(int val) {
    this.val = val;
  }
}*/

public class Solution {

  public ListNode findMiddle(ListNode head) {
    ListNode slow = head;         // Initialize slow pointer at the head of the list
    ListNode fast = head;         // Initialize fast pointer at the head of the list
    while (fast != null && fast.next != null) {  // Traverse the list using slow and fas
      slow = slow.next;           // Move slow pointer one step at a time
      fast = fast.next.next;      // Move fast pointer two steps at a time
    }

    return slow;  // Return the node pointed to by slow (middle node or middle
  }
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The above algorithm will have a time complexity of O(N) where 'N' is the number of nodes in the LinkedList.

Space Complexity

The algorithm runs in constant space O(1).

Please wait! Loading...